



Ursinus
COLLEGE

CS375 Spring 2026

Ralph Rostock

Week 11

Maintenance, Refactoring, and Technical Debt



Ursinus
COLLEGE

Team Standup Meetings



Last Week

CI/CD in Practice: Pipelines, GitHub Actions & Deployment Strategies



What Happens After Release?

“The majority of software cost occurs after initial release – not before.”

- Lientz & Swanson, Software Maintenance Management, 1980

- 60-80% of lifetime cost is post-release (Leintz & Swanson)
- ~ 50% of developer time spent on maintenance
- Between 2x and 4x longer to add features to poorly maintained code

Types of Software Maintenance

Corrective

Fixing bugs and defects after release.
The one everyone notices.

Adaptive

Updating for environmental changes —
new OS, APIs, platforms, regulations.

Perfective

Improving performance, usability,
or adding features users have requested.

Preventive

Refactoring and restructuring to reduce
future risk. Almost always cut first.

Taxonomy: Lientz & Swanson (1980)

In practice: most teams live in corrective and adaptive. Preventive is the first thing cut under deadline pressure — which is exactly when debt compounds fastest.



Refactoring

“Refactoring: improving the internal structure of code without changing its external behavior.”

- Martin Fowler, Refactoring (1999)

- The key constraint: behavior must not change. If it does, that’s a bug fix, not refactoring.
- Tests are not optional – they are the safety net that lets you refactor with confidence.
- Without tests, you are not refactoring – you are hoping.
- The discipline: small steps -> run tests -> commit. Repeat.

The Boy Scout Rule: "Always leave the code a little better than you found it."

Refactoring Techniques

Rename

The most underrated technique. A better name is instant, permanent, free documentation — free, forever. If you need a comment to explain what something does/is, rename it instead.

✗ `calc (x, y, f)`

✓ `calculate_discount (amount, rate, is_flagged)`

Extract Method

Pull a named chunk of logic into its own function. The comment above a code block is usually the name of a function waiting to be born.

✗ `# apply bulk bonus
if s > 500: r += 50`

✓ `apply_bulk_bonus(r, s)`

Replace Magic Number with Constant

Cheapest, safest improvement you can make. Names the concept and creates a single source of truth for the value.

✗ `if retries > 3`

✓ `if retries > MAX_RETRIES`

Three techniques you should be using constantly — cheap, safe, and immediately readable.

Refactoring Techniques

Extract Class

Separate responsibilities into a new, focused class. Each class should have one clear reason to change.

When: one class has too many methods, too many fields, or handles two unrelated concepts.

Introduce Parameter Object

Group related parameters into an object or dataclass. Reduces call-site noise and often names a concept your model was missing.

When: a function takes 4+ arguments, especially ones that often travel together.

Replace Conditional with Polymorphism

Replace type-checking if/else chains with subclasses or a strategy pattern. Adding a new type becomes adding a new class, not editing a chain.

When: long if/else or switch chains check an object's type and do different things for each.

Three techniques for deeper problems – more involved, but the right tool when the design itself is wrong.

Worked Example – Before Refactoring

```
def process(d, t, f, s):  
    if t == 1:  
        r = d * 0.10  
    elif t == 2:  
        r = d * 0.15  
    elif t == 3:  
        r = d * 0.20  
    else:  
        r = 0  
    if f == True:  
        r = r * 2  
    if s > 500:  
        r = r + 50  
    return r
```

What's wrong here? What would you fix?

Worked Example – After Refactoring

Before

```
def process(d, t, f, s):  
    if t == 1:  
        r = d * 0.10  
    elif t == 2:  
        r = d * 0.15  
    elif t == 3:  
        r = d * 0.20  
    else:  
        r = 0  
    if f == True:  
        r = r * 2  
    if s > 500:  
        r = r + 50  
    return r
```

After

```
DISCOUNT_RATES =  
    {1: 0.10, 2: 0.15, 3: 0.20}  
BULK_THRESHOLD = 500  
BULK_BONUS = 50  
  
def calculate_discount(  
    amount, discount_type,  
    is_flagged, sale_amt):  
    rate = DISCOUNT_RATES  
        .get(discount_type, 0)  
    discount = amount * rate  
    if is_flagged:  
        discount *= 2  
    if sale_amt > BULK_THRESHOLD:  
        discount += BULK_BONUS  
    return discount
```

First, rename – parameters and function name now self-document.

Next, replace magic number – constants extracted.

Finally, replace conditional – lookup replaces if/elif.



Legacy Code

“Legacy code is code without tests.”

- *Michael Feathers, Working Effectively with Legacy Code (2004)*

- Age is irrelevant. Code you wrote last week with no tests is already legacy code.
- The real challenge: refactoring code you didn't write, don't fully understand, and that has no tests.
- Without tests, any change is a gamble — you have no way to know what you've broken.

First Move: Characterization Tests

- Before changing anything, write tests that capture what the code currently does — even if you don't know if it's correct.
- These tests are a safety net, not a specification. They tell you when you've accidentally changed behavior.
- Once you have them: refactor with confidence.



When Not to Refactor

Knowing when to hold back is as important as knowing the techniques

✗ Under deadline pressure

Don't start a refactoring you can't finish. A half-refactored codebase is worse than the original. Leave a TODO with context and move on.

✗ Code you don't yet understand

Refactoring requires knowing what behavior you're preserving. Read and understand first, refactor later.

✗ Code with no tests

Write characterization tests first to create a safety net, then refactor. Skipping this is how bugs get introduced.

✗ The rewrite temptation

Joel Spolsky: Throwing away working code is always a mistake. You lose all the implicit bug fixes baked into the old code, and recreate them slowly, plus new ones.



The Team Economics of Technical Debt

Core Problem: Debt is invisible to everyone except the developers carrying it.

What the developer sees:

- Every feature in this module takes 3x as long as it should
- We're afraid to touch the payment code
- We spend 40% of each sprint untangling old decisions
- The codebase is slowly but surely getting harder to work in

What the stakeholder sees:

- Features are slower than expected
- The team seems overly cautious
- Velocity seems inconsistent
- Nothing visible to explain the slowdown

This gap — between what developers experience and what stakeholders see — is where technical debt becomes a team and organizational problem, not just a code problem.



Making the Case to Non-Technical Stakeholders

Translate into terms that matter to the business:

✗ “The code is messy and hard to work with.”

✓ “Every feature in the checkout module takes us 2-3 weeks instead of 3-4 days. Here’s three recent examples.”

✗ “We need to refactor.”

✓ “We’re proposing two weeks of focused work that we estimate will halve our delivery time in this area for the next year.”

✗ “Technical debt is slowing us down.”

✓ “We’re paying roughly 30% interest on every sprint. That interest compounds. Addressing it now costs less than waiting.”

Specifics over adjectives, estimates over feelings, business impact over technical correctness.



Team-Level Practices That Help

If you had to explain to a non-technical sponsor why you need two weeks with no new features, how would you make that case?

- Definition of Done: Include code quality criteria – tests passing, no new code smells introduced, reviews complete. Debt caught at merge is free; debt in production is expensive.
- Code Review as Debt Gate: Reviews are about more than correctness. They are the last checkpoint before new debt enters the codebase. Name what you see.
- Debt as a Backlog Item: Give debt a ticket, an owner, and an estimate like any other work. Invisible debt never gets prioritized; named debt can.
- Blameless Framing: Don't make it personal (e.g., "Who wrote this"). Instead, ask "How do we fix it and prevent more".



Managing Debt Deliberately

Make It Visible

- Debt log or backlog ticket
- TODO comments with context (not just 'TODO: fix this')
- Architecture Decision Records (ADRs) for big tradeoffs
- Complexity metrics and static analysis reports

Allocate Time

- The 20% rule: protect capacity each sprint
- "Refactoring Fridays" or dedicated debt sprints
- Boy Scout commits: small improvements as you go
- Don't promise to fix it – schedule it

Use Your Tools

- Linters: catch surface-level smells automatically
- Complexity metrics (radon, cc or Python)
- Static analysis (SonarQube, CodeClimate)
- Coverage reports as a proxy for untested debt
- **The key principle: debt that is visible and named is manageable. Debt that is invisible is dangerous.**



Key Takeaways – Keeping code healthy over time

- Maintenance is the dominant activity: corrective, adaptive, perfective, preventive.
- Refactoring is disciplined and safe: small moves, tests first, commit often.
- Legacy code isn't old code; it's code without tests. Characterization tests are the first step.
- Know when to stop: deadline, poor understanding, no tests.
- Debt is a team and communication problem, not just a code problem.
- Make it visible, name it, allocate time, and use your tools.

You will inherit other people's debt. You will leave debt for others. The goal is to be a net positive.



Next Week:

How Software Lives in Production, and the Role of AI in Software Engineering



Assignment: Weekly Standup Reflection

<https://rrostock1.github.io/Ursinus-CS375/Assignments/StandupReflection>

(Individual Assignment)

Due Feb 26 (and every week thereafter)



Assignment: The Overdose

<https://rrostock1.github.io/Ursinus-CS375/Assignments/Overdose>

(Individual Assignment)

Due Apr 23



Assignment: Software Test Plan

<https://rrostock1.github.io/Ursinus-CS375/Project/TestPlan>

(Group Assignment)

Due Apr 23